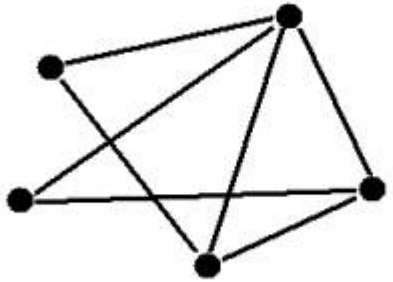
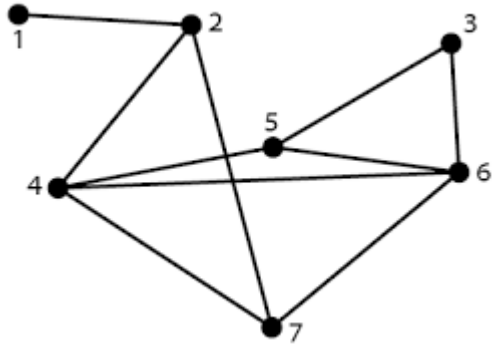
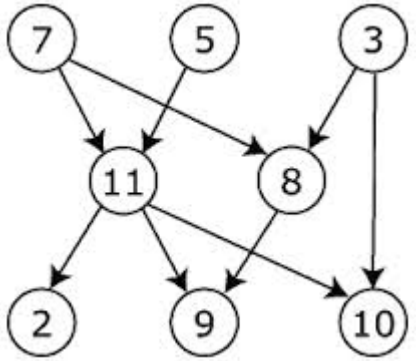
Unidad 7

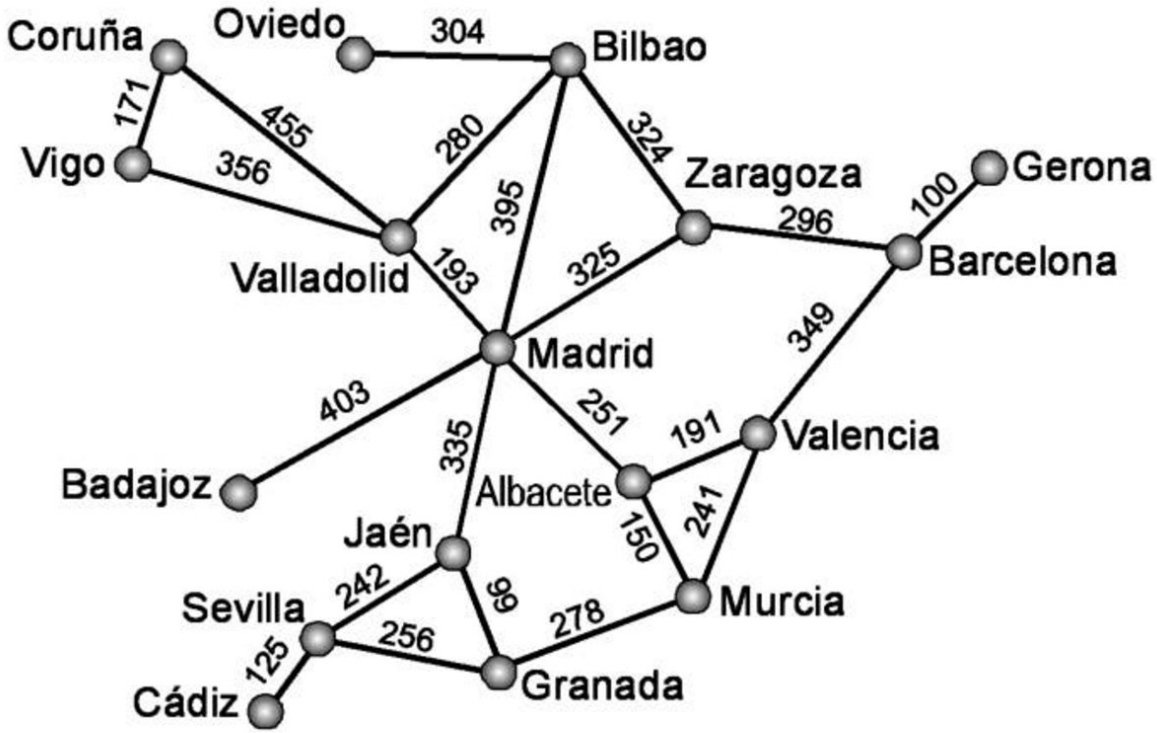
CLASE 7.1 – GRAFOS

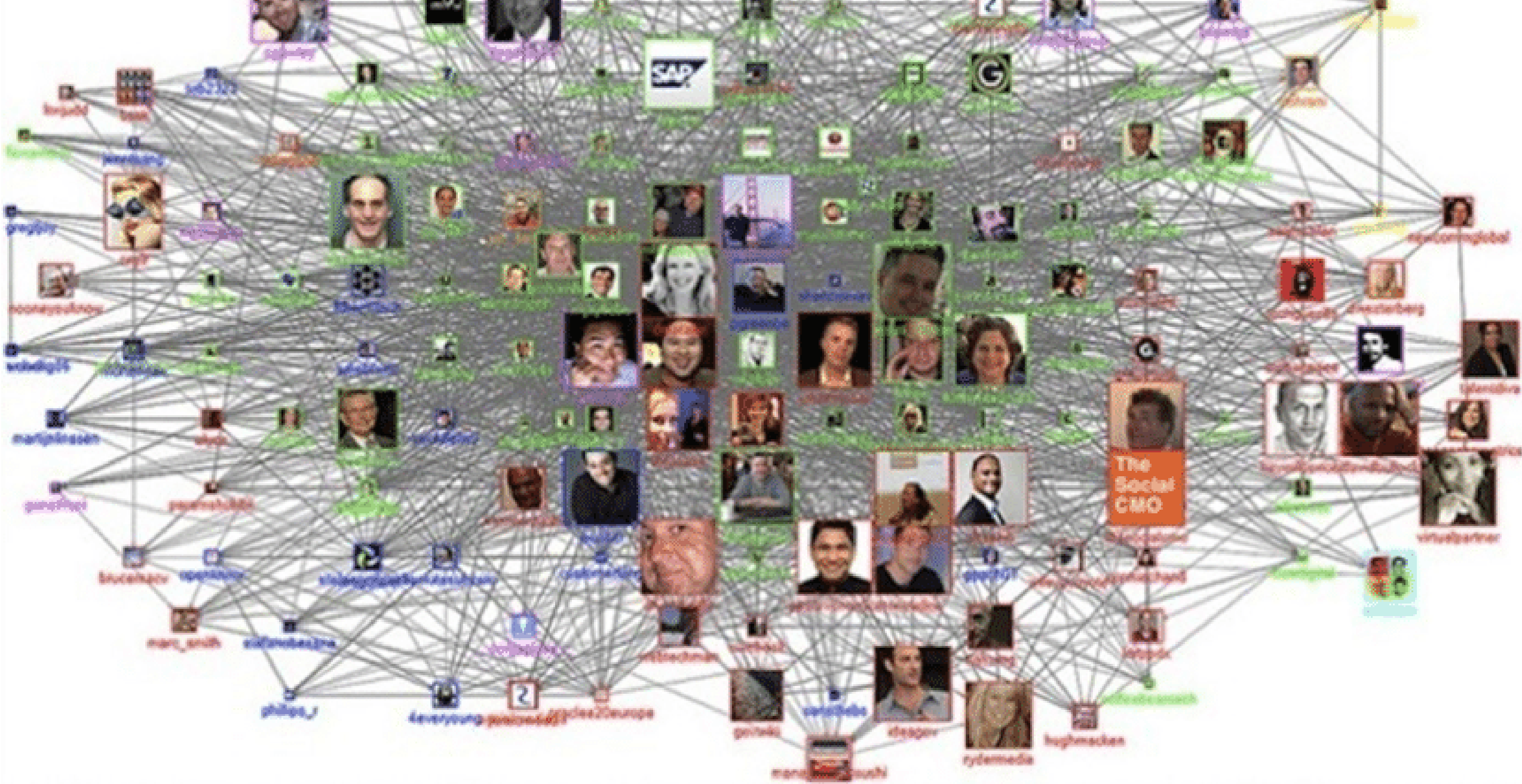
Ejemplos








Coruña
Oviedo
304
Bilbao
171
455
280
324
Vigo
356
Zaragoza
Gerona
100
296
395
193
325
Valladolid
Barcelona
349
Madrid
403
251
191
Valencia
335
Badajoz
Albacete
241
150
Jaén
242
69
Murcia
Sevilla
278
256
125
Granada
Cádiz


The Social CMO

Grafos

Un grafo es un conjunto finito de vértices (nodes) y aristas (edges), donde las aristas unen nodos entre sí.

$G = (V, E)$

Donde:

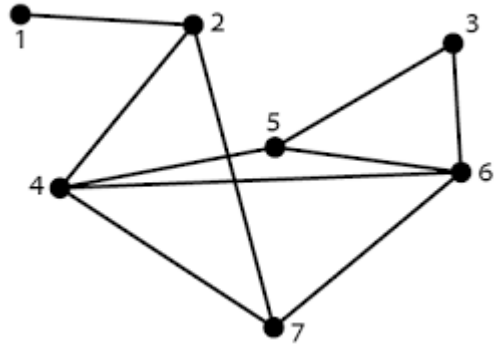
- V es el conjunto de vértices
- E es el conjunto de aristas

Ejemplos

$G_1 = (V_1, E_1)$

$V_1 = \{1,2,3,4,5,6,7\}$

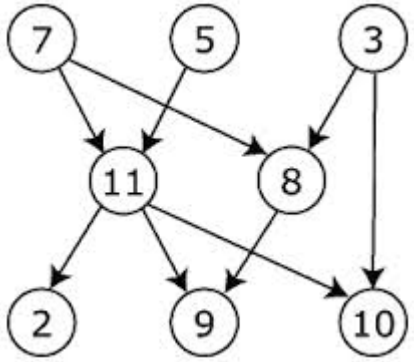
$$E_1 = \left\{ \begin{matrix} \{1,2\}, \{2,4\}, \{2,7\}, \\ \{4,5\}, \{4,6\}, \{4,7\}, \\ \{5,6\}, \{5,3\}, \{3,6\}, \{6,7\} \end{matrix} \right\}$$



$G_2 = (V_2, E_2)$

$V_2 = \{7,5,3,11,8,2,9,10\}$

$$E_2 = \left\{ \begin{matrix} (7,11), (7,8), (5,11), \\ (3,8), (3,10), (8,9), \\ (11,2), (11,9), (11,10) \end{matrix} \right\}$$



Representaciones

Listas de adyacencia

- Más convenientes cuando el grafo es ralo (Sparse)

Matrices de adyacencia

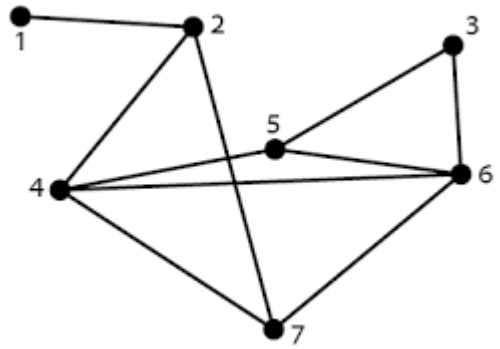
- Más conveniente cuando el número de conexiones aumenta.

Lista de adyacencia

$G_1 = (V_1, E_1)$

$V_1 = \{1,2,3,4,5,6,7\}$

$$E_1 = \begin{Bmatrix} \{1,2\},\{2,4\},\{2,7\}, \\ \{4,5\},\{4,6\},\{4,7\}, \\ \{5,6\},\{5,3\},\{3,6\},\{6,7\} \end{Bmatrix}$$



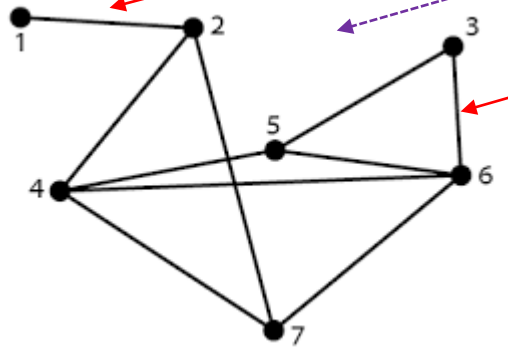

```
graph = dict()
graph['1'] = ['2']
graph['2'] = ['1','4','7']
graph['3'] = ['5','6']
graph['4'] = ['2','5','6','7']
graph['5'] = ['4','6','3']
graph['6'] = ['6','3','5','4','7']
graph['7'] = ['4','2','6']
```

Matriz de adyacencia

$G_1 = (V_1, E_1)$

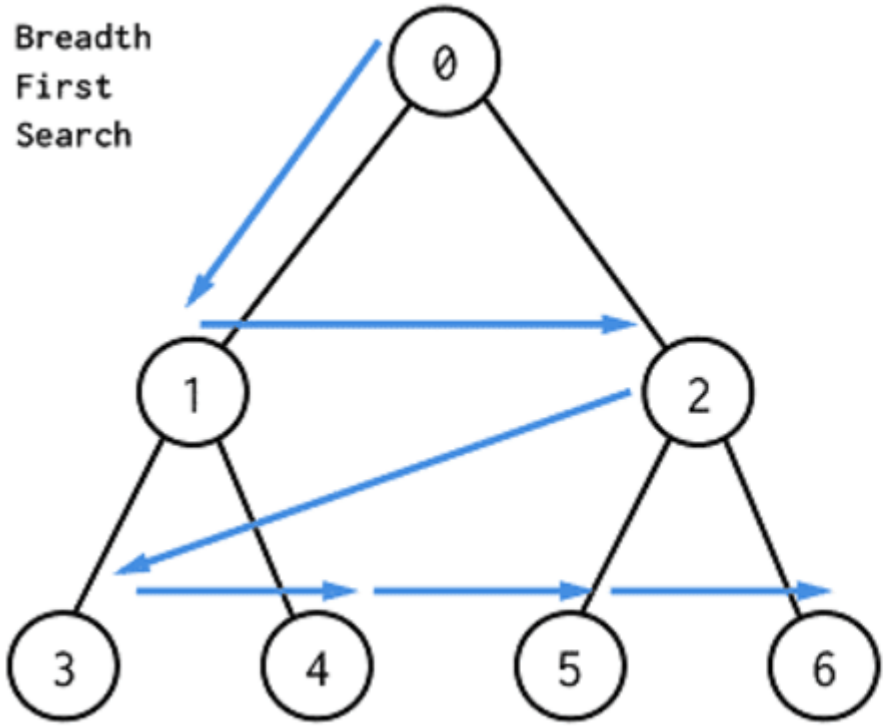
$V_1 = \{1,2,3,4,5,6,7\}$

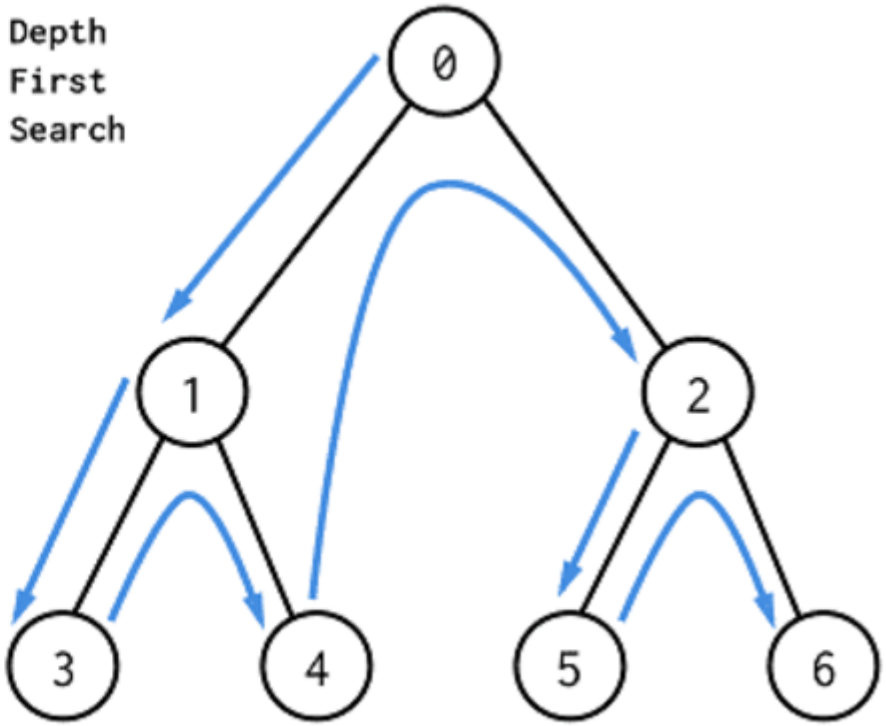
$$E_1 = \begin{Bmatrix} \{1,2\},\{2,4\},\{2,7\}, \\ \{4,5\},\{4,6\},\{4,7\}, \\ \{5,6\},\{5,3\},\{3,6\},\{6,7\} \end{Bmatrix}$$



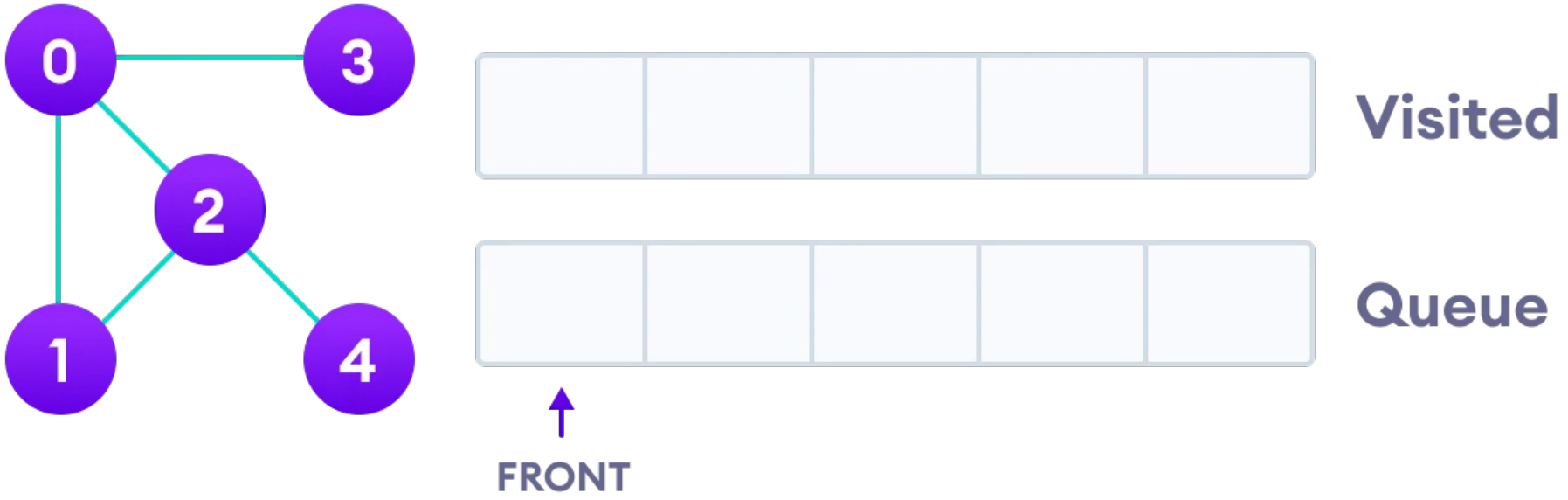
	1	2	3	4	5	6	7
1	0	1	0	0	0	0	0
2	1	0	0	1	0	0	1
3	0	0	0	0	1	1	0
4	0	1	0	0	1	1	1
5	0	0	1	1	0	1	0
6	0	0	1	1	1	0	1
7	0	1	0	1	0	1	0

Recorrido de grafos





BFS (anchura)



```python
# tomado de: https://www.programiz.com/dsa/graph-bfs
# BFS algorithm in Python

import collections

# BFS algorithm
def bfs(graph, root):

    visited, queue = set(), collections.deque([root])
    visited.add(root)

    while queue:

        # Dequeue a vertex from queue
        vertex = queue.popleft()
        print(str(vertex) + " ", end="")

        # If not visited, mark it as visited, and
        # enqueue it
        for neighbour in graph[vertex]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
```